Nama	Annisa Zhafira Mutia Permana
Kelas	TI – 1C
No Absen	04
NIM	264107020111

JOBSHEET 7

Percobaan 1: Studi Kasus Nilai Mahasiswa di SIAKAD – Perulangan FOR

```java
import java.util.Scanner;

public class SiakadFor04 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        double nilai, tertinggi = 0, terendah = 100;
        int lulus = 0, tidakLulus = 0;

        for (int i = 1; i <= 10; i++) {
            System.out.print("Masukkan nilai mahasiswa ke-" + i + ": ");
            nilai = sc.nextDouble();

            if (nilai > tertinggi) {
                tertinggi = nilai;
            }
            if (nilai < terendah) {
                terendah = nilai;
            }

            // Penambahan kondisi kelulusan (minimal 60)
            if (nilai >= 60) {
                lulus++;
            } else {
                tidakLulus++;
            }
        }

        System.out.println("Nilai tertinggi: " + tertinggi);
        System.out.println("Nilai terendah: " + terendah);
        System.out.println("Jumlah mahasiswa lulus: " + lulus);
        System.out.println("Jumlah mahasiswa tidak lulus: " + tidakLulus);

        sc.close();
    }
}
```

```
Masukkan nilai mahasiswa ke-1: 76.5
Masukkan nilai mahasiswa ke-2: 82.3
Masukkan nilai mahasiswa ke-3: 62.1
Masukkan nilai mahasiswa ke-4: 88.4
Masukkan nilai mahasiswa ke-5: 65.9
Masukkan nilai mahasiswa ke-6: 67.9
Masukkan nilai mahasiswa ke-7: 90.1
Masukkan nilai mahasiswa ke-8: 55.3
Masukkan nilai mahasiswa ke-9: 73.7
Masukkan nilai mahasiswa ke-10: 78.6
Nilai tertinggi: 90.1
Nilai terendah: 55.3
Jumlah mahasiswa lulus: 9
Jumlah mahasiswa tidak lulus: 1
```

1. **Sebutkan dan tunjukkan masing-masing komponen perulangan FOR pada kode program Percobaan 1!**

   - Inisialisasi `int i = 1` (Menentukan nilai awal variabel penghitung loop)

- Condition / Kondisi `i <=10` (Syarat agar loop terus berjalan selama selama i kurang dari atau sama dengan 10)
- Increment `i++` (Penambahan nilai variabel `i` sebanyak 1 di setiap akhir iterasi)

2. **Mengapa variabel tertinggi diinisialisasi 0 dan terendah diinisialisasi 100? Apa yang terjadi jika variabel tertinggi diinisialisasi 100 dan terendah diinisialisasi 0?**

   - Tertinggi diinisiasi 0 agar setiap input nilai ujian yang valid (>0) pasti lebih besar dan dapat menggantikan nilai tertinggi awal
   - Terendah diinisiasi 100 agar setiap input nilai (<= 100) pasti lebih kecil dan dapat menggantikan nilai terendah awal
   - Jika dibalik (tertinggi = 100 dan terendah = 0), nilai tertinggi akan tetap 100 dan terendah tetap 0, kecuali jika ada input nilai yang > 100 atau < 0 sehingga lohika pencarian nilai menjadi salah

3. **Jelaskan fungsi dan alur kerja dari potongan kode berikut!**

```
if (nilai > tertinggi) {
    tertinggi = nilai;
}
if (nilai < terendah) {
    terendah = nilai;
}
```

   If (nilai > tertinggi) = memeriksa apakah nilai input lebih besar dari tertinggi, jika true akan menyimpan nilai baru tersebut ke variabel tertinggi.

   If (nilai < terendah) = memerikasa apakah nilai input lebih kecil dari terendah, jika true akan menyimpan nilai baru tersebut ke variabel terendah.

4. **Modifikasi kode program sehingga terdapat perhitungan untuk menentukan berapa mahasiswa yang lulus dan yang tidak lulus berdasarkan batas kelulusan (nilai minimal 60). Tampilkan jumlah mahasiswa lulus dan tidak lulus setelah menampilkan nilai tertinggi dan terendah!**

```java
int lulus = 0, tidakLulus = 0;

for (int i = 1; i <= 10; i++) {
    System.out.print("Masukkan nilai mahasiswa ke-" + i + ": ");
    nilai = sc.nextDouble();

    if (nilai > tertinggi) {
        tertinggi = nilai;
    }
    if (nilai < terendah) {
        terendah = nilai;
    }

    // Penambahan kondisi kelulusan (minimal 60)
    if (nilai >= 60) {
        lulus++;
    } else {
        tidakLulus++;
    }
}

System.out.println("Nilai tertinggi: " + tertinggi);
System.out.println("Nilai terendah: " + terendah);
System.out.println("Jumlah mahasiswa lulus: " + lulus);
System.out.println("Jumlah mahasiswa tidak lulus: " + tidakLulus);
```

```
Masukkan nilai mahasiswa ke-1: 76.5
Masukkan nilai mahasiswa ke-2: 82.3
Masukkan nilai mahasiswa ke-3: 62.1
Masukkan nilai mahasiswa ke-4: 88.4
Masukkan nilai mahasiswa ke-5: 65.9
Masukkan nilai mahasiswa ke-6: 67.9
Masukkan nilai mahasiswa ke-7: 90.1
Masukkan nilai mahasiswa ke-8: 55.3
Masukkan nilai mahasiswa ke-9: 73.7
Masukkan nilai mahasiswa ke-10: 78.6
Nilai tertinggi: 90.1
Nilai terendah: 55.3
Jumlah mahasiswa lulus: 9
Jumlah mahasiswa tidak lulus: 1
```

Percobaan 2: Studi Kasus Nilai Mahasiswa di SIAKAD – Perulangan WHILE



```
Masukkan jumlah mahasiswa: 5
Masukkan nilai mahasiswa ke-1: 85
Nilai mahasiswa ke-1 adalah A
Bagus, pertahankan nilainya
Masukkan nilai mahasiswa ke-2: 63
Nilai mahasiswa ke-2 adalah C+
Masukkan nilai mahasiswa ke-3: 101
Nilai tidak valid. Masukkan lagi nilai yang valid!
Masukkan nilai mahasiswa ke-3: 23
Nilai mahasiswa ke-3 adalah E
Masukkan nilai mahasiswa ke-4: -15
Nilai tidak valid. Masukkan lagi nilai yang valid!
Masukkan nilai mahasiswa ke-4: 70
Nilai mahasiswa ke-4 adalah B
Masukkan nilai mahasiswa ke-5: 55
Nilai mahasiswa ke-5 adalah C
PS C:\Kuliah\Dasar-Pemrograman>
```

1. **Pada potongan kode berikut, tentukan maksud dan kegunaan dari sintaks berikut:**

```
if (nilai < 0 || nilai > 100) {
    System.out.println(x:"Nilai tidak valid. Masukkan lagi nilai yang valid!");
    continue;
}
```

   a. **nilai < 0 || nilai > 100**

      syarat pemeriksaan apakah nilai yang dimasukkan berada di jangkauan valid (kurang dari 0 ATAU lebih dari 100).

b. continue

Menghentikan iterasi saat ini dan langsung melompat kembali ke kondisi pengecekan while sehingga kode pengolahan nilai dibawahnya tidak dieksekusi.

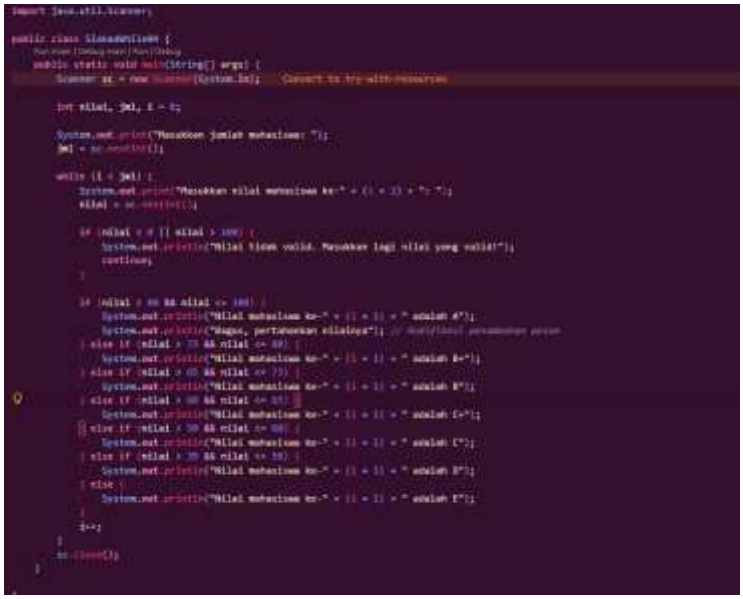
2. **Mengapa sintaks i++ dituliskan di akhir perulangan WHILE? Apa yang terjadi jika posisinya dituliskan di awal perulangan WHILE?**

   Ditulis diakhir agar penghitungan mahasiswa yang valid dilakukan setelah seluruh pengolahan atau ketegori selesai. Jika diletakkan di awal, variabel i langsung bertambah sebelum input diproses, akibatnya penomoran urutan mahasiswa menjadi bergeser (mulai dari 2 bukan 1) dan iterasi akan terhitung bertambah meskipun nilai input tidak valid.

3. **Apabila jumlah mahasiswa yang dimasukkan adalah 19, berapa kali perulangan WHILE akan berjalan?**

   Perulangan while akan berjalan minimal 19 kali (jika semua input valid). Jika terdapat input tidak valid, perualangan akan berjalan lebih dari 19 kali karena instruksi continue mengulang input menambah variabel.

4. **Modifikasi kode program sehingga apabila terdapat mahasiswa yang mendapat nilai A, program menampilkan pesan tambahan "Bagus, pertahankan nilainya"!**



Percobaan 3: Studi Kasus Transaksi di Kafe – Perulangan DO-WHILE



```
Masukkan Nama Pelanggan (Ketik 'batal' untuk keluar) : Rena
Jumlah Kopi : 3
Jumlah Teh : 0
Jumlah Roti : 1
Total Yang Harus Dibayar: Rp 56000
Masukkan Nama Pelanggan (Ketik 'batal' untuk keluar) : Yuni
Jumlah Kopi : 1
Jumlah Teh : 4
Jumlah Roti : 2
Total Yang Harus Dibayar: Rp 80000
Masukkan Nama Pelanggan (Ketik 'batal' untuk keluar) : BATAL
Transaksi Dibatalkan
Semua Transaksi Selesai.
PS C:\Kuliah\Dasar-Pemrograman>
```

1. **Pada penggunaan DO-WHILE ini, apabila nama pelanggan yang dimasukkan pertama kali adalah “batal”, maka berapa kali perulangan dilakukan?**
   Perulangan dilakukan sebanyak 1 kali. Karena do while selalu mengeksekusi blok kode minimal satu kali sebelum memeriksa kondisi atau menemukan perintah break
2. **Sebutkan kondisi berhenti yang digunakan pada perulangan DO-WHILE tersebut!**

Kondisi berhenti karena ada instruksi break; di dalam blok if (namaPelanggan.equalsIgnoreCase(“batal”))

3. **Apa fungsi dari penggunaan nilai true pada kondisi DO-WHILE?**
   Membuat perulangan berjalan tanpa batas (infinite loop) secara bertahap hingga dihentikan secara paksa dari dalam blok perulangan menggunakan perintah break
4. **Mengapa perulangan DO-WHILE tersebut tetap berjalan meskipun tidak ada komponen inisialisasi dan update?**
   Karena kontrol perulangan tidak didasarkan pada perhitungan counter angka seperti i++, melainkan dikendalikan secara eksplisit oleh kondisi logika kustom berbasis string (perintah ”batal”) dan statemen break.

TUGAS 1



```
Masukkan jumlah tiket yang dibeli: 4
Total bayar transaksi ini: Rp 200000.0
Apakah ada transaksi lain? (y/n): n

--- REKAP PENJUALAN HARI INI ---
Total tiket terjual : 4
Total hasil penjualan: Rp 200000.0
PS C:\Kuliah\Dasar-Pemrograman>
```

TUGAS 2

```java
import java.util.Scanner;

public class BayarParkir04 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);    Convert to try-with-resources

        int jenis, durasi;
        int total = 0;

        do {
            System.out.print("Masukkan jenis kendaraan (1 Mobil, 2 Motor, 0 Keluar): ");
            jenis = sc.nextInt();

            if (jenis == 1 || jenis == 2) {
                System.out.print("Masukkan durasi parkir (jam): ");
                durasi = sc.nextInt();

                if (jenis == 1) {
                    total += durasi * 3000;
                } else {
                    total += durasi * 2000;
                }
            }
        } while (jenis != 0);

        System.out.println("Total pembayaran parkir: Rp " + total);

        sc.close();
    }
}
```

```
Masukkan jenis kendaraan (1 Mobil, 2 Motor, 0 Keluar): 2
Masukkan durasi parkir (jam): 8
Masukkan jenis kendaraan (1 Mobil, 2 Motor, 0 Keluar): 1
Masukkan durasi parkir (jam): 7
Masukkan jenis kendaraan (1 Mobil, 2 Motor, 0 Keluar): 0
Total pembayaran parkir: Rp 37000
PS C:\Kuliah\Dasar-Pemrograman>
```